Food Delivery Website

Design Report

Team: xLLM

Members: Fu Rao (24107744), Liu Jing (24107741), Liu Zongrun (24107745), Zhou Jiaxu (24107758)

1 Project Idea

A local food delivery service connects three parties: customers who order meals, several independent restaurants that cook them, and delivery riders who collect and deliver them. The difficulty is not that any one of these actions is hard. It is that the service has to operate while its own facts keep changing, and that the three parties never see the same thing.

Each restaurant controls its own opening times, menu and preparation capacity, and all three change during the day: a dish sells out, the kitchen becomes busy, a shop closes early. An order is only fulfilled if the restaurant and the rider coordinate two separate physical steps — preparing the food and collecting it — and the customer stands outside both of them with no way of knowing what is happening. In practice this produces three recurring failures. The participants duplicate work or wait on each other; the customer telephones the restaurant to ask where the order is, which the restaurant cannot answer either; and the normal path breaks whenever there is a delay, an unavailable dish, a wrong address or a cancellation. There is also a privacy problem underneath: giving all three parties the same view would show one customer another customer's home address, show one restaurant another restaurant's orders, and expose rider contact details to everyone.

The service we will build therefore has to do two things at once: hold the order as a *shared record* that all three parties work on, and give each party a different view of that record according to what they are responsible for.

User roles. We identify three roles. Their needs differ, and so do their permissions — the difference is the core of the design, not an afterthought.

- **Customer.** Needs to order food from a nearby restaurant and to know when it will arrive. May create orders, read and cancel *their own* orders and write a review. Must never see another customer's orders or address, a restaurant's internal workload, or a rider's personal details.
- **Restaurant staff.** Need to receive orders, prepare them, and hand them over. May manage *their own* opening hours, menu items and availability, and move *their own* orders through preparation. Must never see another restaurant's orders or menu, and may not alter the amounts charged on an order.
- **Delivery rider.** Needs to find work and complete deliveries. May see the pool of orders that are prepared and unclaimed, claim one, report position, and move that order through delivery. Must never see orders that are not ready, or orders another rider has claimed.

Why it is useful. The application covers the whole life of an order rather than a single step, and replaces the informal telephone coordination with one shared record that each party can act on, making the exception paths explicit rather than leaving them to improvisation.

2 Main Functions

Functions are listed in the order we intend to build them. The first five are *essential*: without them the topic's central workflow cannot be demonstrated. The remainder are *optional* additions that we will attempt only once the essential set works end to end.

- **F1 — Accounts and roles.** All three roles register and sign in through one entry point; the role determines which interface is shown. Result: an authenticated session that the server uses to authorise every later request.
- **F2 — Restaurant menu and hours.** Restaurant staff set opening hours per weekday, add and edit dishes with price, calories and remaining stock, and mark a dish unavailable. Result: the public menu always reflects what the kitchen can actually produce now.
- **F3 — Browsing and ordering.** A customer browses restaurants, selects a delivery address by pointing at a map, and places an order containing several dishes. Result: an order record holding a *snapshot* of the address and of each dish's name and price.
- **F4 — Order progression.** Restaurant staff accept an order, mark it preparing and then ready; a rider claims the ready order, confirms collection and confirms delivery. Result: the order advances through defined states, and the same record is updated by two different roles in turn.
- **F5 — Order tracking.** The customer sees their own orders and the current state and timestamps of each. Result: progress is visible without contacting the restaurant.
- **F6 — Reviews (optional).** After delivery the customer rates the restaurant and the rider from one to five stars with a comment. Result: a quality signal attached to a completed order.
- **F7 — Rider location (optional).** A rider reports position while delivering and the customer sees it on a map. Result: live progress rather than a status label.
- **F8 — Coupons (optional).** A customer claims and applies a discount at checkout. Result: a discounted total recorded on the order.

Priority. F1–F5 are the minimum for a coherent demonstration: only these are needed to take one order from a request to a delivered outcome through two roles. F6–F8 add value but are not required by the topic, so they come last and are reduced first.

Rules that constrain these functions.

- *Ownership.* A record may only be changed by the party that owns it: a dish by its own restaurant, an order by its own customer, restaurant or rider. The check is made on the server for every query, not by hiding controls in the interface.
- *Transition authority.* Each state change belongs to exactly one role. Only the restaurant may accept and prepare; only a rider may collect and deliver. No role may skip a state.
- *Claim exclusivity.* An order enters the rider pool only when it is ready and has no rider. Claiming is a single conditional update, so if two riders act at the same moment exactly one succeeds and the other is told the order has gone.
- *Cancellation window.* Cancellation is free while the order is still awaiting acceptance. Once the restaurant has accepted, preparation may have started, so cancellation requires the restaurant's agreement.
- *Delivery range.* An order is refused if the selected address lies outside the restaurant's configured delivery radius, and the customer is told why.
- *Record integrity.* Address and prices are copied into the order when it is placed. Later edits to an address or a menu therefore cannot rewrite the history of past orders.

3 Application Design

Application structure. Figure 1 shows the main parts. Three browser interfaces — customer, restaurant and rider — send requests over HTTPS to a reverse proxy, which forwards everything under `/api` to the application server. The application is divided into functional modules, one per business domain. The *authentication module* establishes who the caller is and which role they hold. The *restaurant module* manages shop profiles, opening hours and menu items. The *order module* creates orders and enforces the state machine, and is the only module that writes order states. The *rider module* manages the claimable pool and position reports. The *review module* records ratings against completed orders. All modules share one data access layer and one set of record definitions, so ownership checks are written in the same place for every request. The database holds the records in Table 1. Two external services are used: a map service, called from the browser to capture coordinates and from the server to verify them; and a mail service, used only for password recovery.

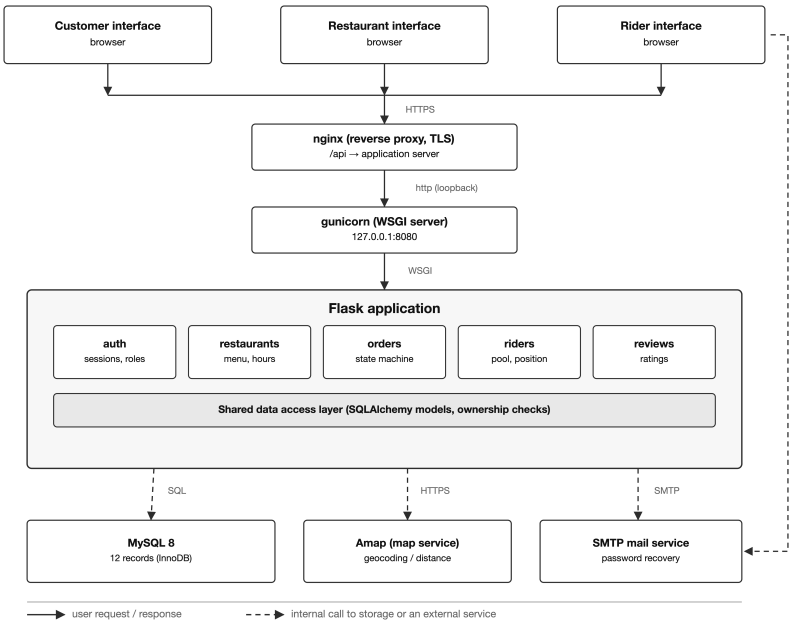


Figure 1: Main parts of the application. Solid arrows carry user requests and responses over HTTPS; the dashed arrows show internal calls to the database and to external services.

Data. Table 1 lists the records the application stores, the information in each, and the functions that use or update them.

How the records connect. An *order* is the record that links the parties: it names the customer who placed it, the restaurant that must cook it and — once claimed — the rider who delivers it, so all three roles act on the same row. An order owns many *order items*, one per dish; each item points at a *menu item* but also carries its own copy of the name and price, which is what allows the menu to change without altering history. A *restaurant* owns its hours and its menu items, and a *customer* owns a list of addresses; the chosen address is copied into the order for the same reason. A *review* is attached to a delivered order and names its target — the restaurant or the rider — and the order alone is what proves the reviewer was entitled to write it. *Rider locations* are appended against an order while it is being delivered.

User workflow. Consider the task of placing and fulfilling one order.

Record	Information stored	Functions that use or update it
Users	Role, username, e-mail, display name, hashed password	F1 creates and authenticates; every other function reads it to decide permissions.
Restaurants	Owner, name, address and coordinates, delivery radius, minimum order, open flag	F2 updates it; F3 reads it to list shops and check the delivery radius.
Opening hours	Restaurant, weekday, open and close time, closed flag	F2 maintains; F3 uses it to refuse orders outside opening hours.
Menu items	Restaurant, name, price, calories, stock, available flag	F2 maintains; F3 reads for display and copies price into the order.
Addresses	Customer, label, contact, address text, coordinates, default flag	F1 owns them; F3 selects one and copies it into the order.
Orders	Customer, restaurant, rider, state, address and amount snapshots, distance, timestamps	F3 creates; F4 and F5 update and read the state; F6 attaches a review to it.
Order items	Order, dish reference, name and price snapshots, quantity	F3 writes them; F4 reads them to show the kitchen what to cook.
Reviews	Order, reviewer, target, rating, comment	F6 creates; F2 and F5 read ratings back.
Rider locations	Rider, order, coordinates, accuracy, timestamp	F7 appends them; F5 reads the latest one for display.
Coupons and claims	Code, owner, discount, threshold, claim status	F8 issues, claims and redeems them against an order.
Verification codes	E-mail, purpose, hashed code, expiry, attempts	F1 sends and checks them when a password is reset.

Table 1: Records, the information stored in each, and the functions that use them.

1. The customer signs in (F1). The authentication module reads *users*, starts a session and returns the role, which selects the customer interface.
2. The customer opens a restaurant (F3). The restaurant module reads *restaurants*, *opening hours* and *menu items* and returns only those items currently available.
3. The customer picks a delivery address on the map and adds dishes to a cart. The map service converts the chosen point into coordinates and a readable address.
4. The customer confirms. The order module checks the distance against the restaurant's radius, then writes one *orders* row with the address and total copied in, plus one *order items* row per dish with name and price copied in. The order starts in state *pending*. Result: the customer can now see progress (F5), and the restaurant can see the order (F4).
5. The restaurant accepts, marks it preparing, then ready. Each step updates the *orders* row and stamps the matching timestamp; only the restaurant that owns the order may do this.
6. The order now appears in the rider pool. A rider claims it — a single conditional update that succeeds only if the order is still ready and still unclaimed — then confirms collection and later delivery. The *orders* row moves to *delivering* and then *delivered*.
7. The customer sees the final state and may leave a review (F6), which is written against this order.

A situation that changes the process: cancellation. Suppose the customer wants to cancel *after* the restaurant has accepted and begun cooking. The normal path offers no free way out, because food has already been committed. The order module therefore does not cancel it directly: it records the request and surfaces it to the restaurant, whose decision it is. If the restaurant agrees, the order

moves to *cancelled* with the cancelling party and reason recorded alongside the timestamp; the order disappears from the restaurant's queue, and if a rider had already claimed it, that rider is released and the order leaves the pool. If the restaurant refuses, the order simply continues along the normal path. The same mechanism covers the reverse timing: while the order is still *pending* nothing has been committed, so the customer cancels immediately and no restaurant action is needed. The rule is thus a property of the order's state, not of the interface the request arrived from.

4 Technology Choices

Browser. HTML5 and CSS3 for structure and presentation; JavaScript with jQuery and AJAX for interaction and for calling the server without reloading the page; Jinja2 templates on the server to render the three role interfaces. The browser is responsible for presentation and input validation only — including the map, which captures the address coordinates.

Server. Python 3.10 with Flask 3.1, using Werkzeug password hashing and Flask sessions, with SQLAlchemy 2.0 as the data access layer. The server is where the application actually lives: it authenticates every request, decides per record whether the caller is entitled to read or change it, runs the order state machine, and is the only part that holds database credentials.

Data storage. MySQL 8 with InnoDB and `utf8mb4`, holding the twelve records in Table 1. A relational store is the right fit because the rules above are relational: ownership is a column on the row, claim exclusivity is *one* conditional `UPDATE` guarded by `rider_id IS NULL`, one-review-per-order is a unique constraint, and per-order history is preserved by copying values rather than joining to live ones. Foreign keys keep the order graph consistent while a record is passed between roles.

Why this combination suits the project. One language for the server and the data layer keeps the ownership checks in one place, which matters because every function depends on them; server-rendered pages with AJAX keep the three interfaces simple while still allowing progress to refresh without a reload; and the relational model expresses the topic's own language — availability, ownership, approval — directly as constraints.

Technology not covered in class. Three items are outside the taught stack and were chosen for specific reasons.

- *Map service (Amap)* — used to let a customer point at a delivery address instead of typing it, and to check the delivery radius. The benefit is direct: hand-typed addresses are the main source of the address problems the topic names, and pointing at a map removes that ambiguity at source. Management: two separate keys are used, a browser key restricted by domain and a server key kept on the server only; the radius check uses straight-line distance computed locally, so the ordering flow still works if the service is unavailable, and the address field falls back to text. Integration has already been carried out and verified end to end.
- *nginx and gunicorn (deployment)* — the reverse proxy terminates HTTPS and forwards requests; gunicorn runs the application. This is an operational rather than a functional choice: Flask's built-in server is explicitly not intended for production, and HTTPS is a prerequisite for the browser location API used in F7. Management: the configuration is fixed and documented; the application code is unchanged.
- *E-mail service (SMTP)* — used only to send a one-time code for password recovery. It is the only out-of-band channel available to us at no cost, since programmatic SMS here requires a paid registered signature and pre-approved templates. Management: sending is isolated so a failure cannot fail the request, and the code's expiry, single use and attempt limits live in our own database, so the security property does not depend on the mail service.